

King Fahad University of Petroleum and Minerals

ICS321

Project Report

Project Name:	Hotel Reservation and Room Management System
Section:	02
Student 1 Name	Jalal Zainaddin
Student 1 ID	202154790
Student 2 Name	Hassan AlMarhoon
Student 2 ID	202467900

Table of Contents

1. Motivation and Problem Statement.....	3
2. List of Probable Stakeholders	3
3. ER Diagram	4
4. Glossary	5
5. Relational Model (Tables)	6
6. Creating Tables and populating data	11
7. User interface used.....	19
8. User Manual.....	20
9. Conclusion	24
10. Work distribution.....	25

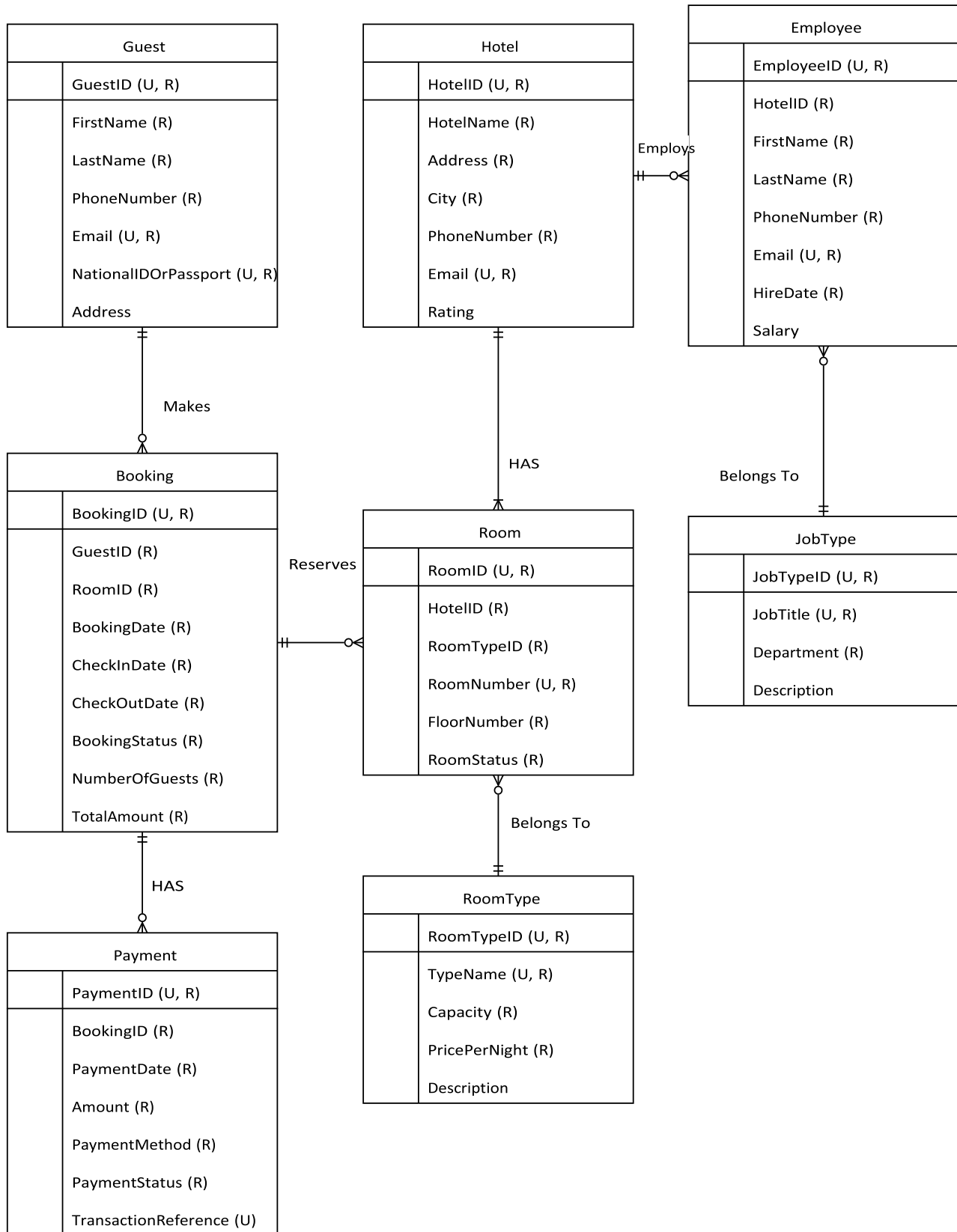
1. Motivation and Problem Statement

Hotels deal with many connected pieces of information at the same time: guest details, room types, room availability, booking dates, payments, and extra services. When this information is handled in a weak or poorly organized database, staff may struggle to find free rooms quickly, update reservations correctly, or generate accurate bills. One booking request can affect several tables at once, and a small error in one part of the system can cause duplicate bookings, inconsistent records, or slow response time. This project focuses on using core database concepts to improve responsiveness and efficiency, especially relational design, primary and foreign keys, normalization, constraints, and indexes. The goal is not only to store data, but to organize it in a way that supports faster queries, cleaner updates, and more reliable hotel operations.

2. List of Probable Stakeholders

Name	Title	Roles in the project	Comments
Hotel Receptionist	Primary User / Front Desk Staff	Uses the system to create bookings, update guest records, assign rooms, check guests in and out, and view room availability.	Main daily user of the database, so the system should be fast, simple, and easy to use.
Hotel Manager	Business Owner / Decision Maker	Reviews occupancy, booking history, room status, and payment records; ensures the system supports hotel operations efficiently.	Interested in accuracy, reporting, and reducing booking errors or duplicate reservations.

3. ER Diagram



4. Glossary

Term	Meaning
Guest	A customer who books or stays in the hotel.
Hotel	The hotel where rooms, employees, and bookings are managed.
Room	A hotel room that can be assigned to a guest booking.
Room Number	The number used to identify a room inside the hotel.
Room Status	The current condition of a room, such as available, occupied, or under maintenance.
Booking	A reservation made by a guest for a room during a specific period.
Booking Date	The date when the reservation was created.
Check-In Date	The date the guest is expected to arrive.
Check-Out Date	The date the guest is expected to leave the hotel.
Booking Status	The current state of a booking, such as pending, confirmed, or cancelled.
Payment	The money paid for a booking.
Payment Date	The date when the payment was made.
Payment Method	The way the payment was made, such as cash, card, or online payment.
Payment Status	The state of the payment, such as paid, pending, or failed.
Employee	A hotel worker who helps operate the hotel system and services.
Job Type	Type of job/work for each hotel employee
Guest Information	Personal details of the guest, such as name, phone number, and email.
Hotel Information	Basic hotel details such as name, address, location, and contact number.
Reservation Period	The full stay duration from check-in date to check-out date.
Total Amount	The total cost of the booking that the guest needs to pay.
Phone Number	A contact number for the guest, hotel, or employee.
Email	An electronic contact address for the guest, hotel, or employee.

5. Relational Model (Tables)

The conceptual ER model of the Hotel Reservation System was converted into a relational model by mapping each entity into a table and each relationship into foreign keys. The resulting schema is designed to support the main hotel operations, including storing guest information, managing rooms and room types, handling employee and job type records, creating bookings, and recording payments.

The relational model was implemented using PostgreSQL as the database management system. PostgreSQL was selected because it supports standard SQL, foreign key constraints, unique constraints, check constraints, and other relational features required for the project. The database was hosted using Aiven for PostgreSQL, which provided an online managed PostgreSQL service that could be accessed from different devices without needing to keep a local database server running all the time. To interact with the database and execute SQL files, the psql command-line tool was used. This tool made it possible to create tables, insert sample data, and run test queries directly against the hosted PostgreSQL database. In addition, Aiven PG Studio was used to visually inspect the tables and verify that the data had been inserted correctly.



1. Hotel

Hotel(
HotelID, HotelName, Address, City, PhoneNumber, Email, Rating
)

Primary Key: HotelID

Stores the main hotel information.

2. Guest

Guest(
GuestID, FirstName, LastName, PhoneNumber, Email, NationalIDOrPassport, Address
)

Primary Key: GuestID

Stores guest personal and contact information.

3. JobType

JobType(
JobTypeID, JobTitle, Department, Description
)

Primary Key: JobTypeID

Defines the employee job category, such as Receptionist or Manager.

4. Employee

Employee(
EmployeeID, HotelID, JobTypeID, FirstName, LastName, PhoneNumber, Email, HireDate, Salary
)

Primary Key: EmployeeID

Foreign Key: HotelID → Hotel(HotelID)

Foreign Key: JobTypeID → JobType(JobTypeID)

Stores employee records and connects each employee to a hotel and a job type.

5. RoomType

RoomType(
RoomTypeID, TypeName, Capacity, PricePerNight, Description
)

Primary Key: RoomTypeID

Stores room categories such as Single and Double.

6. Room

Room(
RoomID, HotelID, RoomTypeID, RoomNumber, FloorNumber, RoomStatus
)

Primary Key: RoomID

Foreign Key: HotelID → Hotel(HotelID)

Foreign Key: RoomTypeID → RoomType(RoomTypeID)

Stores the hotel rooms and links each room to its hotel and room type.

7. Booking

Booking(
BookingID, GuestID, RoomID, BookingDate, CheckInDate, CheckOutDate, BookingStatus,
NumberOfGuests, TotalAmount
)

Primary Key: BookingID

Foreign Key: GuestID → Guest(GuestID)

Foreign Key: RoomID → Room(RoomID)

Stores reservation details and connects each booking to one guest and one room.

8. Payment

Payment(
PaymentID, BookingID, PaymentDate, Amount, PaymentMethod, PaymentStatus,
TransactionReference
)

Primary Key: PaymentID

Foreign Key: BookingID → Booking(BookingID)

Stores payment transactions related to bookings.

Foreign Key Summary

Employee.HotelID → Hotel.HotelID

Employee.JobTypeID → JobType.JobTypeID

Room.HotelID → Hotel.HotelID

Room.RoomTypeID → RoomType.RoomTypeID

Booking.GuestID → Guest.GuestID

Booking.RoomID → Room.RoomID

Payment.BookingID → Booking.BookingID

Relational Design Notes

The relational model was kept simple and practical for a hotel reservation system while still reflecting the important rules from the ER model. Separate tables were used for RoomType and JobType to reduce repetition and improve consistency. The relationships are implemented through foreign keys so that the database preserves valid links between records, such as making sure a booking belongs to an existing guest and room, and a payment belongs to an existing booking.

6. Creating Tables and populating data

The tables were created in an order that respects foreign key dependencies. Independent tables were created first, followed by dependent tables.

SQL Statements for Table Creation:

```
CREATE TABLE Hotel (
    HotelID SERIAL PRIMARY KEY,
    HotelName VARCHAR(100) NOT NULL,
    Address VARCHAR(200) NOT NULL,
    City VARCHAR(100) NOT NULL,
    PhoneNumber VARCHAR(20) NOT NULL,
    Email VARCHAR(100) UNIQUE NOT NULL,
    Rating DECIMAL(2,1)
);

CREATE TABLE Guest (
    GuestID SERIAL PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    PhoneNumber VARCHAR(20) NOT NULL,
    Email VARCHAR(100) UNIQUE NOT NULL,
    NationalIDOrPassport VARCHAR(50) UNIQUE NOT NULL,
    Address VARCHAR(200)
);

CREATE TABLE JobType (
    JobTypeID SERIAL PRIMARY KEY,
    JobTitle VARCHAR(50) UNIQUE NOT NULL,
    Department VARCHAR(100) NOT NULL,
    Description VARCHAR(200)
);

CREATE TABLE RoomType (
    RoomTypeID SERIAL PRIMARY KEY,
    TypeName VARCHAR(50) UNIQUE NOT NULL,
    Capacity INT NOT NULL,
    PricePerNight DECIMAL(10,2) NOT NULL,
    Description VARCHAR(200)
);

CREATE TABLE Employee (
    EmployeeID SERIAL PRIMARY KEY,
    HotelID INT NOT NULL,
    JobTypeID INT NOT NULL,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    PhoneNumber VARCHAR(20) NOT NULL,
    Email VARCHAR(100) UNIQUE NOT NULL,
    HireDate DATE NOT NULL,
    Salary DECIMAL(10,2),
    CONSTRAINT fk_employee_hotel
        FOREIGN KEY (HotelID) REFERENCES Hotel(HotelID),
```

```

        CONSTRAINT fk_employee_jobtype
        FOREIGN KEY (JobTypeID) REFERENCES JobType(JobTypeID)
    );

CREATE TABLE Room (
    RoomID SERIAL PRIMARY KEY,
    HotelID INT NOT NULL,
    RoomTypeID INT NOT NULL,
    RoomNumber VARCHAR(10) NOT NULL,
    FloorNumber INT NOT NULL,
    RoomStatus VARCHAR(20) NOT NULL,
    CONSTRAINT fk_room_hotel
        FOREIGN KEY (HotelID) REFERENCES Hotel(HotelID),
    CONSTRAINT fk_room_roomtype
        FOREIGN KEY (RoomTypeID) REFERENCES RoomType(RoomTypeID),
    CONSTRAINT uq_room_per_hotel UNIQUE (HotelID, RoomNumber),
    CONSTRAINT chk_room_status
        CHECK (RoomStatus IN ('Available', 'Occupied', 'Maintenance',
'Cleaning'))
);

CREATE TABLE Booking (
    BookingID SERIAL PRIMARY KEY,
    GuestID INT NOT NULL,
    RoomID INT NOT NULL,
    BookingDate DATE NOT NULL,
    CheckInDate DATE NOT NULL,
    CheckOutDate DATE NOT NULL,
    BookingStatus VARCHAR(20) NOT NULL,
    NumberOfGuests INT NOT NULL,
    TotalAmount DECIMAL(10,2) NOT NULL,
    CONSTRAINT fk_booking_guest
        FOREIGN KEY (GuestID) REFERENCES Guest(GuestID),
    CONSTRAINT fk_booking_room
        FOREIGN KEY (RoomID) REFERENCES Room(RoomID),
    CONSTRAINT chk_booking_dates
        CHECK (CheckOutDate > CheckInDate),
    CONSTRAINT chk_booking_guests
        CHECK (NumberOfGuests > 0),
    CONSTRAINT chk_total_amount
        CHECK (TotalAmount >= 0),
    CONSTRAINT chk_booking_status
        CHECK (BookingStatus IN ('Pending', 'Confirmed', 'Cancelled',
'Checked-In', 'Checked-Out'))
);

CREATE TABLE Payment (
    PaymentID SERIAL PRIMARY KEY,
    BookingID INT NOT NULL,
    PaymentDate DATE NOT NULL,
    Amount DECIMAL(10,2) NOT NULL,
    PaymentMethod VARCHAR(20) NOT NULL,
    PaymentStatus VARCHAR(20) NOT NULL,
    TransactionReference VARCHAR(100) UNIQUE,
    CONSTRAINT fk_payment_booking
        FOREIGN KEY (BookingID) REFERENCES Booking(BookingID),
    CONSTRAINT chk_payment_amount

```

```
    CHECK (Amount > 0),  
    CONSTRAINT chk_payment_method  
    CHECK (PaymentMethod IN ('Cash', 'Card', 'Online')),  
    CONSTRAINT chk_payment_status  
    CHECK (PaymentStatus IN ('Pending', 'Paid', 'Failed', 'Refunded'))  
);
```

Enforced business rules

- A room status must be one of: Available, Occupied, Maintenance, Cleaning.
- Check-out date must be later than check-in date.
- Number of guests must be greater than zero.
- Payment amount must be greater than zero.
- Booking and payment statuses must come from predefined valid lists.
- Room number must be unique within the same hotel.

One important business rule is that the same room should not be booked for overlapping dates. This rule is logically part of the project, but it is usually handled at the application or procedural level instead of with a simple SQL constraint.

SQL Statements for Sample Data

```
INSERT INTO Hotel (HotelName, Address, City, PhoneNumber, Email, Rating)
VALUES
('Palm Stay Hotel', 'King Fahd Road', 'Dhahran', '0130000000',
'info@palmstay.com', 4.5);

INSERT INTO Guest (FirstName, LastName, PhoneNumber, Email,
NationalIDOrPassport, Address)
VALUES
('Ahmed', 'Ali', '0501234567', 'ahmed@example.com', 'S1234567', 'Dammam'),
('Sara', 'Hassan', '0559876543', 'sara@example.com', 'S7654321', 'Khobar');

INSERT INTO JobType (JobTitle, Department, Description)
VALUES
('Receptionist', 'Front Desk', 'Handles guest bookings and reception'),
('Manager', 'Administration', 'Supervises hotel operations');

INSERT INTO RoomType (TypeName, Capacity, PricePerNight, Description)
VALUES
('Single', 1, 200.00, 'Single bed room'),
('Double', 2, 350.00, 'Double bed room');

INSERT INTO Employee (HotelID, JobTypeID, FirstName, LastName, PhoneNumber,
Email, HireDate, Salary)
VALUES
(1, 1, 'Mona', 'Khalid', '0541111111', 'mona@palmstay.com', '2026-05-01',
5000.00),
(1, 2, 'Omar', 'Salem', '0542222222', 'omar@palmstay.com', '2026-05-03',
8500.00);

INSERT INTO Room (HotelID, RoomTypeID, RoomNumber, FloorNumber, RoomStatus)
VALUES
(1, 1, '101', 1, 'Available'),
(1, 2, '102', 1, 'Occupied');

INSERT INTO Booking (GuestID, RoomID, BookingDate, CheckInDate, CheckOutDate,
BookingStatus, NumberOfGuests, TotalAmount)
VALUES
(1, 1, '2026-05-03', '2026-05-10', '2026-05-12', 'Confirmed', 1, 400.00),
(2, 2, '2026-05-04', '2026-05-15', '2026-05-18', 'Checked-In', 2, 1050.00);

INSERT INTO Payment (BookingID, PaymentDate, Amount, PaymentMethod,
PaymentStatus, TransactionReference)
VALUES
(1, '2026-05-03', 400.00, 'Card', 'Paid', 'TXN1001'),
(2, '2026-05-04', 1050.00, 'Cash', 'Paid', 'TXN1002');
```

Testing the Database

After inserting the sample data, test queries were executed to confirm that the schema and relationships were functioning correctly.

```
SELECT * FROM Hotel;
SELECT * FROM Guest;
SELECT * FROM JobType;
SELECT * FROM Employee;
SELECT * FROM RoomType;
SELECT * FROM Room;
SELECT * FROM Booking;
SELECT * FROM Payment;
```

Output:

Command Line (PostgreSQL):

hotelid	hotelname	address	city	phonenumber	email	rating
1	Palm Stay Hotel	King Fahd Road	Dhahran	0130000000	info@palmstay.com	4.5

(1 row)

guestid	firstname	lastname	phonenumber	email	nationalidorpassport	address
1	Ahmed	Ali	0501234567	ahmed@example.com	S1234567	Dammam
2	Sara	Hassan	0559876543	sara@example.com	S7654321	Khobar

(2 rows)

jobtypeid	jobtitle	department	description
1	Receptionist	Front Desk	Handles guest bookings and reception
2	Manager	Administration	Supervises hotel operations

(2 rows)

employeeid	hotelid	jobtypeid	firstname	lastname	phonenumber	email	hiredate	salary
1	1	1	Mona	Khalid	0541111111	mona@palmstay.com	2026-05-01	5000.00
2	1	2	Omar	Salem	0542222222	omar@palmstay.com	2026-05-03	8500.00

(2 rows)

roomtypeid	typename	capacity	pricepernight	description
1	Single	1	200.00	Single bed room
2	Double	2	350.00	Double bed room

(2 rows)

roomid	hotelid	roomtypeid	roomnumber	floornumber	roomstatus
1	1	1	101	1	Available
2	1	2	102	1	Occupied

(2 rows)

bookingid	guestid	roomid	bookingdate	checkindate	checkoutdate	bookingstatus	numberofguests	totalamount
1	1	1	2026-05-03	2026-05-10	2026-05-12	Confirmed	1	400.00
2	2	2	2026-05-04	2026-05-15	2026-05-18	Checked-In	2	1050.00

Aiven Online DB:

The screenshot shows the Aiven Online DB console interface. The top navigation bar includes the Aiven logo, 'Home', 'Projects', 'Tools', 'Billing', and 'Admin'. The breadcrumb trail indicates the current location: 'My Organization / hotel-reservation-3ba8 / hotel-3b715a3b / PG Studio'. The left sidebar contains a list of navigation items: 'Back to project', 'Overview', 'Integrations', 'Metrics', 'PG Studio', 'AI insights', 'Logs', 'Query statistics', 'Current queries', 'Users', 'Databases', 'Connection pools', 'Backups', and 'Service settings'. The main content area is divided into three sections. The top section is the 'Editor' tab, showing a query editor with the text 'SELECT * FROM hotel;'. The middle section is the 'Results' tab, displaying the query results in a table. The table has 7 columns: 'address', 'city', 'email', 'hotelid', 'hotelname', 'phonenummer', and 'rating'. The first row of data is: 'King Fahd Road', 'Dhahran', 'info@palmstay.com', '1', 'Palm Stay Hotel', '0130000000', and '4.5'. The bottom section is the 'Saved queries' tab, which is currently empty.

Hotel-RESERVATION-3BA8
hotel-3b715a3b

My Organization / hotel-reservation-3ba8 / hotel-3b715a3b / PG Studio [Early availability](#)

Editor |←| Untitled query x +

Open schema map

Did you like this tool?

1 SELECT *
2 FROM hotel;

No saved queries.
Save a query for easy accessibility by clicking the button

Source: defaultdb Schema: public 00:00:00.118 Save Run

Results Elapsed time: 0.118s Read: 1 row (172 B) Search for...

address	city	email	hotelid	hotelname	phonenummer	rating
King Fahd Road	Dhahran	info@palmstay.com	1	Palm Stay Hotel	0130000000	4.5

Example join query for booking details

SELECT

```
b.BookingID,
g.FirstName,
g.LastName,
r.RoomNumber,
b.CheckInDate,
b.CheckOutDate,
b.BookingStatus
```

FROM Booking b

```
JOIN Guest g ON b.GuestID = g.GuestID
```

```
JOIN Room r ON b.RoomID = r.RoomID;
```

```
psql (18.3, server 17.9)
WARNING: Console code page (437) differs from Windows code page (1252)
         8-bit characters might not work correctly. See psql reference
         page "Notes for Windows users" for details.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.

defaultdb=> SELECT
defaultdb->   b.BookingID,
defaultdb->   g.FirstName,
defaultdb->   g.LastName,
defaultdb->   r.RoomNumber,
defaultdb->   b.CheckInDate,
defaultdb->   b.CheckOutDate,
defaultdb->   b.BookingStatus
defaultdb-> FROM Booking b
defaultdb-> JOIN Guest g ON b.GuestID = g.GuestID
defaultdb-> JOIN Room r ON b.RoomID = r.RoomID;
 bookingid | firstname | lastname | roomnumber | checkindate | checkoutdate | bookingstatus
-----
1 | Ahmed    | Ali      | 101        | 2026-05-10 | 2026-05-12 | Confirmed
2 | Sara     | Hassan   | 102        | 2026-05-15 | 2026-05-18 | Checked-In
3 | Ahmed    | Ali      | 101        | 2026-05-10 | 2026-05-12 | Confirmed
4 | Sara     | Hassan   | 102        | 2026-05-15 | 2026-05-18 | Checked-In
(4 rows)
```

The screenshot shows the Aiven console interface. On the left is a sidebar with navigation links: Overview, Integrations, Metrics, PG Studio, AI insights, Logs, Query statistics, Current queries, Users, Databases, Connection pools, Backups, and Service settings. The main area displays a SQL query in the 'Editor' tab. The query is a JOIN query for booking details. Below the query editor, there are controls for source (defaultdb) and schema (public), a timer (00:00:00.291), and buttons for 'Save' and 'Run'. The 'Results' section shows the output of the query, which is a table with 7 columns: bookingid, bookingstatus, checkindate, checkoutdate, firstname, lastname, and roomnumber. The results show 4 rows of data.

bookingid	bookingstatus	checkindate	checkoutdate	firstname	lastname	roomnumber
1	Confirmed	2026-05-10	2026-05-12	Ahmed	Ali	101
2	Checked-In	2026-05-15	2026-05-18	Sara	Hassan	102
3	Confirmed	2026-05-10	2026-05-12	Ahmed	Ali	101
4	Checked-In	2026-05-15	2026-05-18	Sara	Hassan	102

Final Implementation Summary

The relational model of the Hotel Reservation System was successfully implemented using SQL in PostgreSQL. All tables were created with the required keys and constraints, and sample data was inserted to simulate realistic hotel operations. The resulting database supports core hotel reservation functions, including guest management, room management, employee records, bookings, and payments.

7. User interface used

To make the Hotel Reservation and Room Management System accessible and usable, a web-based frontend interface was developed using HTML, CSS, and JavaScript. The interface allows guests to browse available rooms, make reservations, and view their booking history. It communicates directly with the PostgreSQL database hosted on Aiven through a backend API, providing a complete end-to-end system.

The front end was built using HTML5 and styled with Tailwind CSS, a pre-made CSS framework that enabled a consistent UI design. JavaScript was used to handle user interactions, form submissions, and dynamic content rendering. The interface consists of several pages:

1. **Home Page:** Displays the hotel overview and allows guests to explore available room types.
2. **Login and Registration Page:** Allows guests to log in using their name and email. Guest information is verified against the database, and the session is stored in the browser using localStorage.
3. **Rooms Page:** Fetches all available rooms from the database and displays them as cards with type, price, floor, and availability status.
4. **Booking Page:** Allows a logged-in guest to select check-in and check-out dates, view a price breakdown, and confirm a reservation. On confirmation, a new booking record is inserted into the database, and the room status is updated.
5. **Reservation History Page:** Displays all past and current bookings for the logged-in guest, with the ability to cancel a confirmed reservation, which updates both the booking status and the room availability in the database.

The front end does not connect to the PostgreSQL database directly. Instead, a backend API was built using Node.js and the Express framework to act as the bridge between the web interface and the database. The back end exposes a set of restful API endpoints that the front end calls using the browser's built-in fetch API.

The backend uses the pg library to connect to the Aiven-hosted PostgreSQL database using a secure connection string stored in an environment variable. All database queries, including guest login verification, room retrieval, booking insertion, and status updates, are executed on the server side and the results are returned to the front end as JSON responses.

CORS (Cross-Origin Resource Sharing) was configured on the backend to allow requests from the frontend served by the Live Server development tool.

The following technologies were used to build and connect the user interface to the database:

1. **HTML5 and Tailwind CSS:** Used to build and style web pages.
2. **JavaScript (Vanilla):** Used for frontend logic, API calls, and session management via localStorage.
3. **Node.js and Express:** Used to build the backend REST API that connects the frontend to the database.
4. **pg (node-postgres):** The Node.js library used to connect to and query the PostgreSQL database.

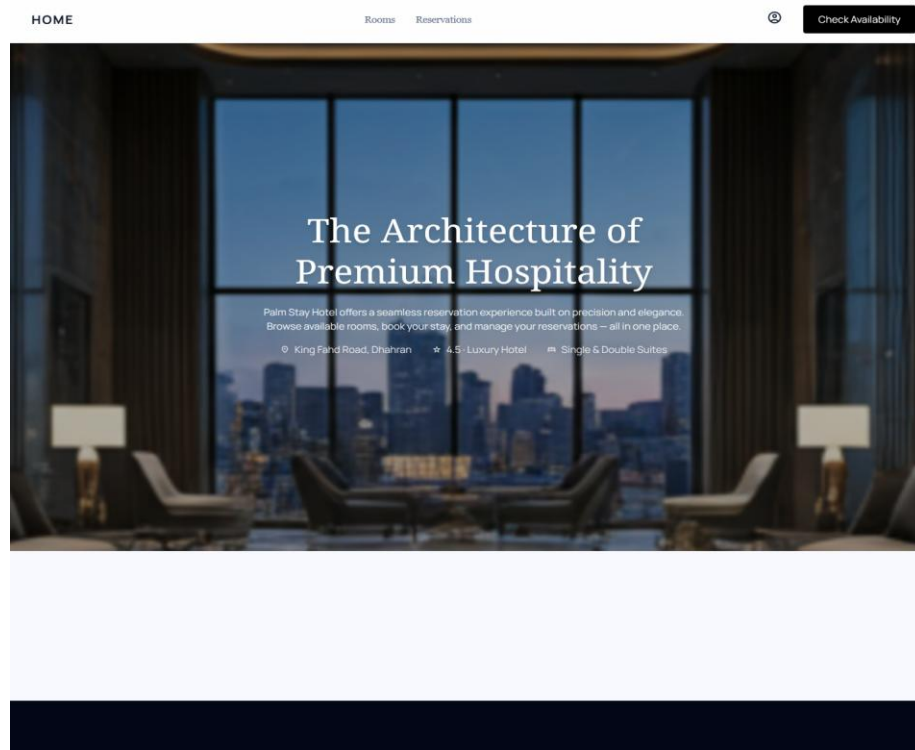
5. Aiven for PostgreSQL: The cloud-hosted database that stores all hotel, guests, rooms, booking, and payment data.

8. User Manual

This section provides a step-by-step guide on how to use the Hotel Reservation and Room Management System. Each step is accompanied by a screenshot of the corresponding page.

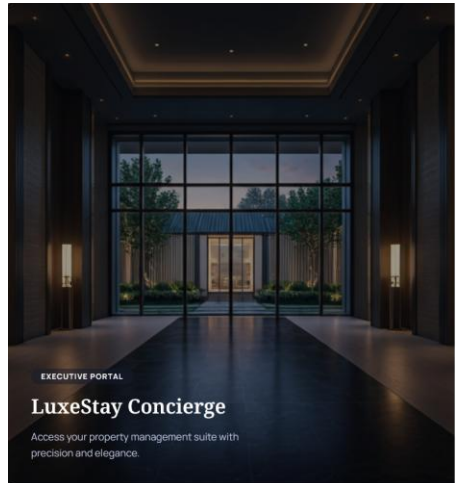
Step 1: Home Page

When the system is opened, the guest is greeted with the home page. The page displays the hotel name, a brief description, and navigation links to browse rooms or view reservations. A login button is available in the top navigation bar.



Step 2: Logging In

To access booking features like viewing the customer history and book a room, the guest must log in. Clicking the account icon in the navigation bar opens the login page. The guest enters their name and registers email address, then clicks the login button. If the credentials match a record in the database, the guest is redirected to the home page and their session is saved.



EXECUTIVE PORTAL

LuxeStay Concierge

Access your property management suite with precision and elegance.

LUXESTAY CONCIERGE

Welcome Back

Please enter your credentials to manage your estate.

FULL NAME

WORK EMAIL

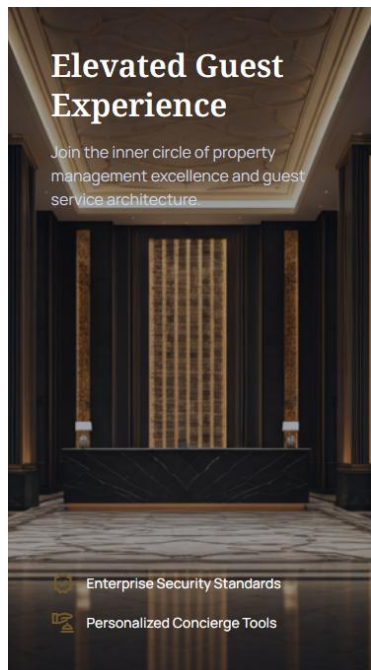
[Login to Dashboard](#)

NEW TO LUXESTAY?

Need access to the property? [Register for an Account](#)

Step 3: Register account

In case you don't have an active account you can register one from the text in the bottom the login page. The required data for registration is first name, last name, phone number, email address, and a residential address.



Elevated Guest Experience

Join the inner circle of property management excellence and guest service architecture.

[Enterprise Security Standards](#)

[Personalized Concierge Tools](#)

Create Account

Provide your details to access the management portal.

First Name

Last Name

Phone Number

Email Address

National ID or Passport

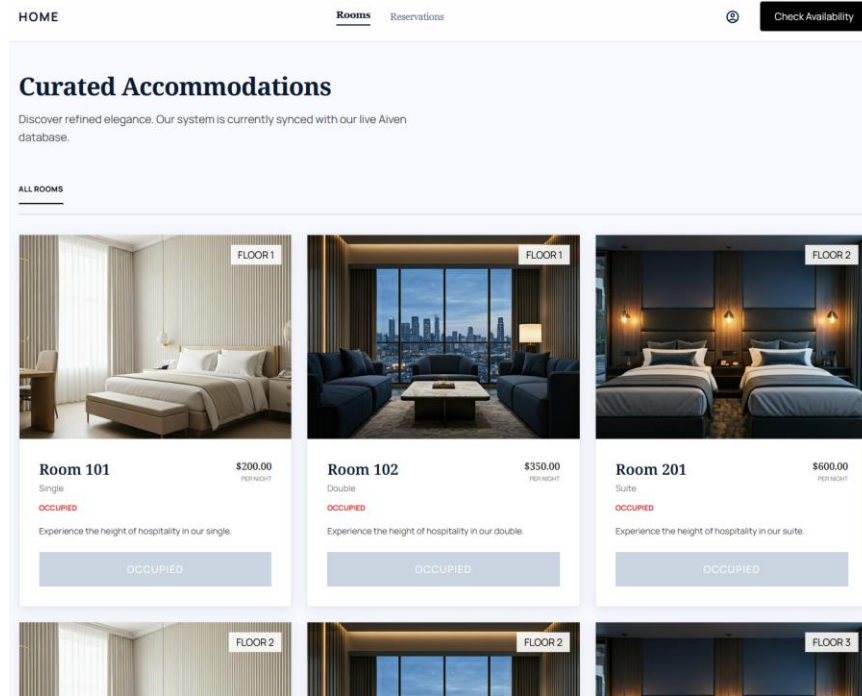
Residential Address

[CREATE ACCOUNT](#)

Already have an account? [Login here](#)

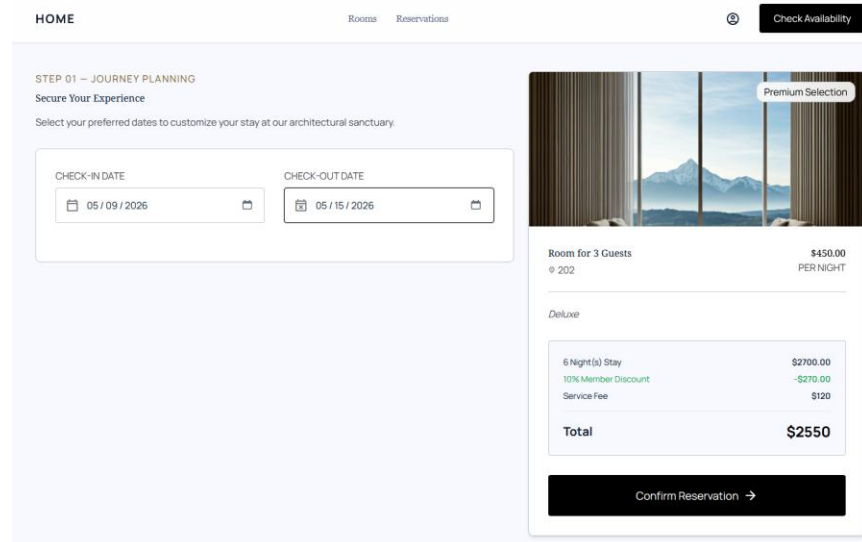
Step 4: Browsing Available Rooms

After logging in, the guest can navigate to the Rooms page. All available rooms are displayed as cards showing the room type, price per night, floor number, and current availability status. Rooms marked as Occupied or ready for booking. The guest clicks on an available room to proceed to the booking page.



Step 5: Making a Reservation

On the booking page, the guest selects a check-in date and a check-out date. The system automatically calculates the total cost based on the number of nights and the room's price per night, including a 10% member discount and a service fee. Once the guest is satisfied with the details, they click the Confirm Booking button. The reservation is saved to the database and the room status is updated to Occupied.



Step 6: Viewing Reservation History

The guest can view all their past and current reservations by navigating to the Reservations page. Each booking is displayed as a card showing the room number, check-in and check-out dates, number of

guests, total amount, and booking status. Confirmed bookings show a Cancel Reservation button, while cancelled and completed bookings are shown for reference only.

HOME Rooms **Reservations** [Check Availability](#)

Reservation History

Review and manage your past and future stays at LuxeStay properties.

Search by ID or Room Name Filter By Wing All Stays Upcoming Completed



ROOM 202
LuxeStay Reservation ID: RSK-30

CHECK-IN May 9, 2026	CHECK-OUT May 15, 2026	GUESTS 3	TOTAL \$2550.00
-------------------------	---------------------------	-------------	--------------------

CONFIRMED [Cancel Reservation](#)



ROOM 302
LuxeStay Reservation ID: RSK-29

CHECK-IN May 9, 2026	CHECK-OUT May 11, 2026	GUESTS 2	TOTAL \$750.00
-------------------------	---------------------------	-------------	-------------------

CONFIRMED [Cancel Reservation](#)

Step 7: Cancelling a Reservation

To cancel a booking, the guest clicks the Cancel Reservation button on a confirmed booking card in the Reservation History page. A confirmation prompt appears asking the guest to confirm the cancellation. Once confirmed, the booking status is updated to Cancelled in the database.



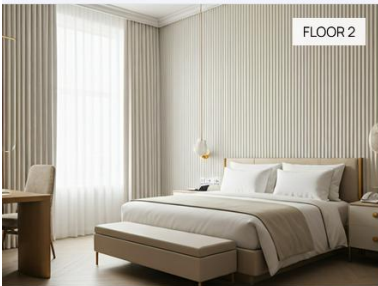
ROOM 202
LuxeStay Reservation ID: RSK-30

CHECK-IN May 9, 2026	CHECK-OUT May 15, 2026	GUESTS 3	TOTAL \$2550.00
-------------------------	---------------------------	-------------	--------------------

CANCELLED [Cancelled](#)

and the room is made available again for other guests.

FLOOR 2



Room 202
Deluxe

\$450.00
PER NIGHT

AVAILABLE

Experience the height of hospitality in our deluxe.

[BOOK NOW](#)

9. Conclusion

This project's purpose is to implement a Hotel Reservation and Room Management System using core database concepts. Starting from a real-world problem, the system was built from the ground up, beginning with an ER diagram that captured the key entities and relationships, followed by a normalized relational model that was implemented in PostgreSQL and hosted on the Aiven cloud platform.

The database schema enforces important business rules through constraints such as check constraints on booking and room statuses, foreign key relationships that preserve data integrity across tables, and unique constraints that prevent duplicate records. These design decisions ensured that the database behaves reliably and consistently under real hotel operation conditions.

A fully functional web-based user interface was developed to bring the database to life. Using HTML, Tailwind CSS, and JavaScript on the frontend, and Node.js with Express on the backend, guests can log in, browse available rooms, make reservations, and manage their bookings through a clean and intuitive interface. All frontend interactions trigger real database operations through a REST API, ensuring that the data displayed to the user always reflects the true state of the system.

Overall, this project demonstrated how a well-structured relational database, combined with a practical application layer, can effectively solve the challenges of hotel reservation management. The system supports accurate room availability tracking, prevents double bookings, and provides guests with a seamless experience from browsing to booking to cancellation. The work completed here forms a strong foundation that could be extended in the future with more features to make it strongly reliable and dependable.

10. Work distribution

1- Jalal Zainaddin:

Made ER diagram design, relational model and SQL table creation with all constraints, sample data and test queries, and Aiven database setup.

2- Hassan AlMarhoon:

Made front end design with HTML/Tailwind/JS, and made the database queries in Node.js for back end connection to database.